REGISTRADORES E O FORMATO DA INSTRUÇÃO · ENCONTRO 2 · LIGA PROTOFPGA

# Gabarito

## Parte A — Atividade em aula

| Item | Resposta |
|---|---|
| 2 | `$zero` = 0, `$at` = 1, `$v0` = 2, `$t0` = 8, `$t7` = 15, `$s0` = 16, `$t8` = 24, `$sp` = 29, `$ra` = 31. Repare que `$t8` não vem logo depois de `$t7`: os temporários foram divididos em dois grupos. |
| 3 | add: `0x01095020` sub: `0x01095822`. Mudaram dois dígitos: o sexto (0 → 8) e o último (0 → 2). |
| 4 | `000000 01000 01001 01011 00000 100010` → op 0 · rs 8 (`$t0`) · rt 9 (`$t1`) · rd 11 (`$t3`) · shamt 0 · funct 34 (`sub`). |
| 5 | Mudaram rd (10 → 11) e funct (32 → 34). O funct mudou porque a conta é outra; o rd mudou porque o resultado foi guardado em outro registrador, `$t3` em vez de `$t2`. Erro mais comum: responder que só o funct mudou. |
| 6 | `0x2150FFFF` → `001000 01010 10000 1111111111111111` → op 8 (`addi`) · rs 10 (`$t2`) · rt 16 (`$s0`) · imediato −1. O destino está em **rt**. Erro mais comum: ler o imediato como 65 535, esquecendo o complemento de dois. |
| 7 | op 0 · rs 12 (`$t4`) · rt 13 (`$t5`) · rd 14 (`$t6`) · shamt 0 · funct 37 → `000000 01100 01101 01110 00000 100101` → `0x018D7025`. |
| 8 | O MARS mostra `0x018d7025`. Quem chegou em `0x01CC6825` pôs o destino primeiro nos bits (rd, rs, rt) — é o erro mais comum da aula. Quem procurou `or` na tabela de op esqueceu que ele é formato R. |
| 9 | `$zero` continua 0. O montador aceita a linha (vira `addiu $0,$0,7`), mas o registrador 0 está ligado ao zero no circuito e a escrita é descartada. Repare que nem o destaque de “registrador alterado” aparece. |
| 10 | Duas linhas, em dois endereços: `lui $1,1` e `ori $17,$1,0x86a0`. O registrador que aparece sem ter sido escrito é o `$1` — o `$at` —, que termina valendo 65 536 (`0x00010000`). 100 000 não cabe em 16 bits, e o montador o usa como rascunho. |
| 11 | `$t2` = −2 147 483 648 (`0x80000000`); `$t3` continua 0. O `add` detectou o transbordo e o MARS parou com *Runtime exception at 0x00400010: arithmetic overflow*. A fórmula de encerramento nunca chegou a ser executada. |

## Parte B — Desafio da semana

| Code | Campos | Instrução |
|---|---|---|
| `0x24080007` | `001001 00000 01000 0000000000000111` | `addiu $t0, $zero, 7` (ou `li $t0, 7`) |
| `0x2409FFFD` | `001001 00000 01001 1111111111111101` | `addiu $t1, $zero, -3` (ou `li $t1, -3`) |
| `0x01095022` | `000000 01000 01001 01010 00000 100010` | `sub $t2, $t0, $t1` |
| `0x01488020` | `000000 01010 01000 10000 00000 100000` | `add $s0, $t2, $t0` |
| `0x01498824` | `000000 01010 01001 10001 00000 100100` | `and $s1, $t2, $t1` |

Valores finais: `$t0` = 7, `$t1` = −3, `$t2` = 7 − (−3) = 10, `$s0` = 10 + 7 = 17, `$s1` = 8.

O último é o que pede os bits. −3 em 32 bits é `1111…1101`; 10 é `0000…1010`. Fazendo E bit a bit, só sobrevive o bit que vale 1 nos dois:

```
   0000 ... 0000 1010   (10)
E  1111 ... 1111 1101   (−3)
=  0000 ... 0000 1000   ( 8)
```

O programa digitado no MARS:

```
        .text
        .globl main
main:
        li   $t0, 7
        li   $t1, -3
        sub  $t2, $t0, $t1
        add  $s0, $t2, $t0
        and  $s1, $t2, $t1
        li   $v0, 10
        syscall
```

Quem trocou a ordem das origens (`and $s1, $t1, $t2`) chegou ao mesmo valor, mas a um código diferente (`0x012A8824`) — e a conferência da coluna Code acusa. Esse é justamente o objetivo do passo 3: o resultado certo não basta; os bits precisam bater.

## Parte C — Exercícios de revisão

**1.** Porque o campo que identifica um registrador na instrução tem 5 bits, e 5 bits formam $2^5 = 32$ combinações, de `00000` a `11111`. Cada combinação nomeia um registrador.

**2.** Falso. Para o circuito, `$s0` e `$t0` são idênticos; a divisão em temporários e salvos é uma convenção entre programadores. A partir do encontro 5, desrespeitá-la passa a causar problemas — mas por acordo, não por proibição do hardware.

**3.** Continua 0. A soma é calculada, mas o registrador 0 está ligado ao zero no circuito, e o resultado é descartado.

**4.** No campo **rd**, bits 15 a 11 — o terceiro registrador nos bits, depois de rs e rt. No texto ele vem primeiro; nos bits, por último.

**5.** Pelo campo **funct**: `100000` (32) é soma, `100010` (34) é subtração. O op `000000` só diz que a instrução é de formato R.

**6.** Mudou só o **imediato** (bits 15–0): `0000000000001100` (12) passou a `0000000000010100` (20). O destino, no formato I, é o campo **rt** (`01000` = `$t0`), que não mudou — assim como op e rs.

**7.** De −32 768 a 32 767. O imediato tem 16 bits em complemento de dois; com n bits, a faixa vai de $-2^{n-1}$ a $2^{n-1} - 1$, ou seja, de $-2^{15}$ a $2^{15} - 1$.

**8.** 5 = `0000 0000 0000 0101`; invertendo: `1111 1111 1111 1010`; somando 1: `1111 1111 1111 1011` = `0xFFFB`.

**9.** `0x012A8824` = `0000 0001 0010 1010 1000 1000 0010 0100` → `000000 01001 01010 10001 00000 100100` → op 0 (R) · rs 9 (`$t1`) · rt 10 (`$t2`) · rd 17 (`$s1`) · shamt 0 · funct 36 (`and`) → `and $s1, $t1, $t2`.

**10.** rs = `$s0` = 16 = `10000`; rt = `$t0` = 8 = `01000`; rd = `$s2` = 18 = `10010`; funct = `100010` → `000000 10000 01000 10010 00000 100010` → `0000 0010 0000 1000 1001 0000 0010 0010` = `0x02089022`.

**11.** `0x7FFFFFFF` + 1 = `0x80000000`, que em complemento de dois vale −2 147 483 648: dois positivos deram negativo, ou seja, houve transbordo. `addu` não verifica e guarda o resultado em `$t2`; `add` verifica, detecta o transbordo e o MARS interrompe o programa com *arithmetic overflow*, sem gravar `$t3`. É exatamente a diferença entre `addi` e `addiu` citada no encontro 1: o **u** não quer dizer “sem sinal”, quer dizer “sem verificação de transbordo”.

### Sobre os códigos de máquina deste gabarito

O MARS escreve os códigos com letras minúsculas (`0x2150ffff`); este gabarito usa maiúsculas em algumas passagens. É a mesma coisa. Se a coluna Basic mostrar `0xffffffff` no lugar de `-1`, é só a exibição: desmarque *Settings* → *Values displayed in hexadecimal*.